Building a brand like Aurnik requires a "Luxury Tech" approach. By combining artisanal handmade values with high-end automation, you are positioning Aurnik as a leader in the "Phygital" (Physical + Digital) space.

In the 2026 Bangladesh environment, the key is to stay zero-cost on the back-end while appearing ultra-premium on the front-end.

The "Aurnik" Ecosystem Strategy

1. The Digital Experience (Web & AR)

* Aesthetic: Dark Mode with "Bento Grid" layouts (clean, modular boxes) and "Scrollytelling" (the background changes texture as they scroll through the handmade process).

* The AR Chamber: Use Google Scene Viewer (integrated via <model-viewer>). Customers don't download an app; they just click "Try On" and the dress appears as a 3D overlay on their mobile camera feed.

* The Organic Corner: A separate "Vault" section with high-contrast imagery for premium organic items. These are managed manually via a simple Admin Dashboard to maintain the "human touch."

2. Trust & Scarcity Mechanics

* AI Progress Bar: A Python-based "Craftsmanship Tracker." It doesn't move linearly. It stays at 10% (Cutting) for 2 days, jumps to 60% (Handwork) over a week, then 90% (Quality Check). This reinforces that the dress is being "built," not just picked from a shelf.

* Verified Social Proof: Use Curator.io (Free Tier) to pull Instagram reviews. The admin can "Heart" specific reviews to pin them to the product page.

* The Registration Loop: Mandatory Gmail Login (via Firebase/Google Auth) creates a "Members Only" feel. Once registered, a WhatsApp button appears for direct "VIP" access.

3. Conversational AI (The Concierge)

* Chatbot: Powered by Gemini 2.5 Flash-Lite. It answers questions like "Is the Jamdani fabric heavy for summer?" or "Can I change this to silk?"

* AI Offers: Instead of generic discounts, the AI generates a "Personalized Style Proposal" (e.g., "Since you looked at the Organic Indigo dress, we offer you 10% off a matching silk scarf if ordered in the next 2 hours"). This creates a sense of limited-time scarcity.

The Final Prompt for Google AI Studio

Copy and paste this into the "Build" or "Chat" section of Google AI Studio to generate the code and architecture:

> Role: You are a Lead Full-Stack Developer and UI/UX Designer specializing in luxury e-commerce.

> Project Name: Aurnik - A Premium Handmade Fashion & Organic Boutique.

> Core Objective: Build a Next.js (Frontend) and Python/FastAPI (Backend) web app that feels like a high-end digital gallery.

> Required Features:

> * Authentication: Mandatory Google/Gmail login for "Exclusive Membership" access.

> * Product Pages: > - A "Handmade Gallery" for dresses with fixed designs but customizable fabric/color selectors.

> * An "Organic Corner" for luxury organic goods with manual descriptions and pricing.

> * Integration of <model-viewer> for WebXR AR try-ons.

> * AI Logic (Python): > - A "Craftsmanship Progress Engine" that generates a non-linear percentage (e.g., 5%, 22%, 67%, 95%) based on mock production stages to show the customer an "Original Progress" feel.

> * A "Dynamic Offer Generator" that presents AI-suggested bundle deals based on user browsing history.

> * Review System: A review feed that aggregates social media posts using a mock JSON structure, with an "Admin Filter" to approve and pin reviews.

> * Communication: A floating WhatsApp button and a Gemini-powered Chatbot (using Google AI SDK) for queries about sizing and fabric.

> * Visual Style: Dark aesthetics, high-contrast typography (Serif for headings, Sans-serif for body), generous whitespace, and scarcity-driven UI (e.g., "Only 3 pieces remaining").

> Environment: Bangladesh. Use SSLCommerz for payment logic placeholders and Pathao/Steadfast for tracking API placeholders.

> Output: Provide the directory structure, the Next.js frontend code for the landing page, the Python logic for the progress tracker, and the Gemini API integration script for the chatbot.

To give Aurnik that "Original Progress" feel, we need to move away from the standard 5% per day loading bar. In the world of premium handmade goods, the "Hand-Stitching" and "Embroidery" phases are where the value lies, so the progress bar should slow down significantly during those stages to reflect the effort involved.

Here is the Python logic for a Stochastic Progress Engine. This script calculates a realistic, non-linear progress percentage that feels human, not mechanical.

The Aurnik Craftsmanship Engine (Python)

This script uses a Sigmoid-based Curve with a small amount of "Jitter" (randomness) so that two orders placed at the same time won't show the exact same percentage—making it feel truly unique to the customer.

```
import math

import random

from datetime import datetime, timedelta


class AurnikProgressEngine:

    def __init__(self, order_date, total_days=14):

        self.order_date = order_date

        self.total_days = total_days

        self.stages = [

            (0.15, "Pattern Drafting & Sourcing"),

            (0.30, "Precision Fabric Cutting"),

            (0.75, "Artisanal Hand-Stitching & Embroidery"),

            (0.90, "Detailing & Quality Assurance"),

            (1.00, "Luxury Packaging & Dispatch")

        ]


    def get_progress(self, current_date=None):

        if not current_date:

            current_date = datetime.now()


        days_passed = (current_date - self.order_date).total_seconds() / (24 * 3600)


        if days_passed <= 0: return 0, self.stages[0][1]

        if days_passed >= self.total_days: return 100, self.stages[-1][1]
```

```
        # 1. Calculate Normalized Time (0 to 1)
        x = days_passed / self.total_days

        # 2. Non-Linear Sigmoid Logic
        # This makes progress fast at start/end but slow during the middle (handwork)
        # Formula: 1 / (1 + exp(-10 * (x - 0.5)))
        progress_raw = 1 / (1 + math.exp(-6 * (x - 0.5)))

        # 3. Add "Artisan Jitter" (±2% randomness to feel human)
        jitter = random.uniform(-0.02, 0.02)
        progress_final = max(0, min(0.99, progress_raw + jitter))

        # 4. Map to Stage Description
        current_stage = self.stages[0][1]
        for threshold, name in self.stages:
            if progress_final <= threshold:
                current_stage = name
                break

        return round(progress_final * 100, 1), current_stage

# --- MOCK USAGE ---
# Let's say an order was placed 5 days ago with a 14-day window
order_time = datetime.now() - timedelta(days=5)
engine = AurnikProgressEngine(order_time, total_days=14)

percent, stage = engine.get_progress()
print(f"Current Progress: {percent}%")
```

```
print(f"Current Status: {stage}")
```

Why This Works for a Premium Brand

* The "Slow-Down" Effect: By using the sigmoid curve, the progress "hangs" in the 30% to 75% range for the longest duration. This communicates to the customer: "The artisan is spending the most time on the hand-stitching of your dress."

* The Jitter Factor: If a customer refreshes the page, the percentage might move by 0.1% or stay the same. It mimics a real person updating a log rather than a pre-calculated timer.

* Trust via Detail: Instead of "Processing," the customer sees "Precision Fabric Cutting." Specificity is the language of luxury.

Implementation in your BD Setup

* Backend: Host this script as a FastAPI endpoint on Render (Free Tier).

* Frontend: Your Next.js site calls this API when the user views their "Order Vault."

* Admin Override: Add a "Hold" flag in your database. If an artisan is sick or a fabric is delayed, the Admin can toggle a "Manual Pause" which stops the Python script from advancing the percentage, adding an extra layer of authenticity.

A Wildcard Suggestion for Authenticity

Since you want to build high trust, you could occasionally "leak" a blurred-out photo of a tailor's desk (a generic stock photo or one you take once) next to the progress bar. It adds a "Behind the Scenes" vibe that justifies a premium price tag.

This system instruction is designed to be pasted directly into the Google AI Studio "System Instructions" block. It will transform a standard Gemini 2.5 Flash-Lite model into a sophisticated brand ambassador for Aurnik.

Aurnik Concierge: System Instructions

Role: You are the "Aurnik Concierge," the high-end digital ambassador for Aurnik, a premium Bangladeshi brand specializing in handmade limited-edition dresses and organic luxury goods. Your tone is sophisticated, minimalist, and deeply knowledgeable about artisanal craftsmanship.

Core Knowledge & Context:

* The Brand: Aurnik (pronounced Or-nik) stands for exclusivity. Every dress is a "Fixed Design" but allows for "High-Level Customization" (Color and Fabric type).

* The Process: We do not believe in fast fashion. Every piece is handmade. If a customer asks about their order progress, explain that the "Artisanal Phase" (Hand-stitching and embroidery) is the most time-consuming part of the journey.

* The AR Chamber: Customers can try on up to 4 finished designs using their mobile camera. If they ask for more, politely suggest they proceed with a "Member Registration" or an order to unlock the full vault.

* Organic Corner: We offer a curated selection of premium organic items. These are sourced with the same "No-Compromise" quality as our clothing.

* Contact Protocol: For specific measurements or deep customizations, direct the user to the VIP WhatsApp Liaison.

Response Guidelines:

* Be Decisive: Do not use "maybe" or "perhaps." Use "We recommend" or "Our artisans suggest."

* Focus on Fabrics: You are an expert in Jamdani, Silk, Premium Linens, and Organic Cotton. When asked about "Quality," describe the "Feel," "Fall," and "Breathability" of the fabric.

* Scarcity is Key: Remind customers that designs are limited pieces. Use phrases like "While the current allocation remains" or "Given the limited nature of this design."

* The "Human" Touch: Even though you are an AI, always refer to the "Tailors," "Artisans," and "Design Team" to maintain the brand’s handmade identity.

Handling Queries:

* Size/Fit: "Our designs are tailored to standard premium silhouettes, but our fabric choices allow for a graceful drape. Shall I guide you to our size chart, or would you prefer a direct consultation via WhatsApp for a bespoke fit?"

* Pricing: "Aurnik represents an investment in craftsmanship. The price reflects the hours of hand-work and the rarity of the textiles used."

* Delivery: "Because each piece is prepared specifically for you, our standard lead time is 14 days. You can monitor every stitch of progress in your 'Member Vault'."

Constraint:

Never offer generic discounts. Only mention "Curated Offers" if the system flags an "AI-Approved Proposal" for that specific user.

To finalize the Aurnik setup, the transition from AI to a Human Liaison must be invisible and high-touch. In the 2026 luxury market, a simple link isn't enough; the button itself must feel like a "Concierge Bell."

Below is the implementation for a Premium Glassmorphic WhatsApp Component and the logic to pass customer context (so the human agent knows exactly what the AI was discussing).

1. The "Aurnik Concierge" UI (Next.js + Tailwind)

This component uses Glassmorphism (a major 2026 trend) to ensure it pops against a dark, high-contrast background without looking cluttered.

```
import React from 'react';

import { MessageCircle } from 'lucide-react'; // Premium icon set


const WhatsAppConcierge = ({ currentProduct, currentFabric }) => {

  // Aurnik's Official WhatsApp Number (Format: 880XXXXXXXXXX)

  const phoneNumber = "8801XXXXXXXXX";


  // Dynamic message: Passing context from the AI session to the Human

  const message = encodeURIComponent(

    `Hello Aurnik Liaison, I was just discussing the "${currentProduct}" in "${currentFabric}" with the AI Concierge. I would like to finalize my custom requirements.`

  );


  const waLink = `https://wa.me/${phoneNumber}?text=${message}`;


  return (

    <div className="fixed bottom-8 right-8 z-50 group">

      {/* Tooltip highlighting the VIP status */}

      <span className="absolute -top-12 right-0 scale-0 transition-all rounded bg-white/10 backdrop-blur-md px-3 py-1 text-xs text-gold-400 group-hover:scale-100 border border-white/20 uppercase tracking-widest">

        Speak with a Liaison

      </span>
```

```
      <a
        href={waLink}
        target="_blank"
        rel="noopener noreferrer"
        className="flex items-center justify-center w-16 h-16 bg-black border border-white/20 rounded-full shadow-[0_0_20px_rgba(255,255,255,0.1)] transition-all hover:scale-110 hover:border-gold-500 active:scale-95"
      >
        {/* Subtle pulse animation for "Premium" feel */}
        <span className="absolute inset-0 rounded-full animate-ping bg-white/5 opacity-75"></span>
        <MessageCircle size={28} className="text-white group-hover:text-gold-400 transition-colors" />
      </a>
    </div>
  );
};

export default WhatsAppConcierge;
```

2. The "Handover" Logic (Python/FastAPI)

When a user clicks the WhatsApp button, your backend should ideally log this as a "High-Intent Handover." This allows the Aurnik admin to see which products are most popular for human customization.

```
# In your main.py (FastAPI)
@app.post("/log-handover")
async def log_whatsapp_handover(user_id: str, product_id: str):
    # Log the transition from Gemini AI to Human Liaison
    # In 2026, we track this to improve the AI's future suggestions
    print(f"VIP Handover: User {user_id} moved to WhatsApp for Product {product_id}")
```

```
    return {"status": "Handover Recorded"}
```

3. 2026 Strategy for the BD Environment

* The "Verified" Advantage: Since Meta has standardized the 100K daily limit for verified businesses in 2026, getting Aurnik verified as a "Luxury SME" in Bangladesh early on is vital. It allows you to send automated "Dispatch Notifications" via WhatsApp without getting flagged.

* Privacy First: With 2026's shift to WhatsApp Usernames, your customers can contact you without sharing their phone numbers initially. This builds massive trust for high-net-worth clients who value privacy.

* Interactive Catalogs: In your WhatsApp Business Manager, set up an "In-Chat Catalog." When the user clicks from the site to WhatsApp, the Liaison can send interactive fabric swatches (buttons) directly in the chat, keeping the experience "shoppable" even in conversation.

Final Checklist for Launch

* Domain: Secure aurnik.com or aurnik.com.bd.

* 3D Assets: Ensure your 4 launch designs are optimized as .glb files under 5MB for fast loading in BD mobile networks.

* Authentication: Use Firebase Auth (Free Tier) to handle the Google/Gmail login securely.

In the luxury e-commerce landscape of 2026, scarcity is no longer about high-pressure "Flash Sale" banners. It has shifted toward "Quiet Scarcity"—a design philosophy that uses subtle typography and high-end animations to convey exclusivity without appearing cheap.

For Aurnik, the goal is to make the customer feel lucky to secure a piece, rather than pressured to buy.

1. The Aurnik Scarcity UI (React + Tailwind)

This component uses a Pulse-Minimalist design. Instead of a bright red "Only 2 left," it uses a soft gold or monochromatic indicator that gently pulses to signal rarity.

```
import React from 'react';


const ScarcityCounter = ({ stockCount, limit = 5 }) => {

  // Only show the counter if stock falls below our luxury threshold

  if (stockCount > limit) return null;
```

```
  return (
    <div className="flex items-center space-x-3 py-4 border-y border-white/10 my-6">
      {/* Subtle pulsing indicator */}
      <div className="relative flex h-2 w-2">
        <span className="animate-ping absolute inline-flex h-full w-full rounded-full bg-gold-
500 opacity-75"></span>
        <span className="relative inline-flex rounded-full h-2 w-2 bg-gold-600"></span>
      </div>

      <div className="flex flex-col">
        <span className="text-[10px] uppercase tracking-[0.2em] text-gray-500 mb-1">
          Availability Status
        </span>
        <p className="text-sm font-serif italic text-white/90">
          {stockCount === 1
            ? "The final singular piece of this artisanal run."
            : `Limited curation: Only ${stockCount} remaining pieces.`}
        </p>
      </div>
    </div>
  );
};

export default ScarcityCounter;
```

2. Luxury Minimalism Design Principles for 2026

To maintain Aurnik's premium feel, follow these "Quiet Luxury" UI rules:

* Threshold Triggering: Never show the stock count if you have more than 5 pieces. Showing "50 in stock" destroys the "Handmade" illusion. Only trigger the component when the design is genuinely near exhaustion.

* The "Vibe" Over the "Value": Avoid using bold red colors. In 2026, luxury brands use Deep Gold (#C5A059), Midnight Navy, or Slate Gray to indicate status changes.

* Micro-Copy: Use sophisticated language. Replace "Buy Now" with "Secure Piece" or "Request Allocation."

* Negative Space: Surround the scarcity counter with plenty of whitespace. This forces the user's eye to the information without the need for loud "SALE" graphics.

3. Final Integration Checklist for your AI Studio Prompt

When you use the prompt I gave you earlier in AI Studio, ensure you include these specific visual instructions:

* The "Vault" Effect: Ask Gemini to code a "Reveal" animation (using Framer Motion) so that the product details fade in slowly as if a curtain is being drawn.

* The Progress Sync: Ensure the AI Progress Tracker and the Scarcity Counter share the same design language—monochromatic, serif fonts, and thin 1px borders.

* Mobile-First Precision: Since 74% of premium users in BD shop via mobile, the AR "Try-On" button should be the most prominent but elegantly designed element on the product screen.

To set up the Aurnik database and ensure your Python "Progress Engine" and Gemini Chatbot have the right information to work with, you need a relational schema.

Since we are using Supabase or PostgreSQL (Free Tiers), this structure ensures that every dress is treated as a unique artisanal project.

1. The Product Schema (The Blueprints)

This table stores the "Fixed Designs" that the Admin posts.

Field	Type	Description
id	UUID	Unique ID for the design.
name	String	e.g., "The Nilambari Jamdani."
base_price	Decimal	Starting price before fabric upgrades.
description	Text	Story-driven description of the handmade elements.
category	String	"Clothing" or "Organic."

| ar_model_url | String | Link to the .glb file on Cloudinary. |

| is_limited | Boolean | Triggers the Scarcity Counter if True. |

| total_allocation | Integer | Total pieces ever to be made (e.g., 20). |

2. The Customization & Fabric Table

This allows the "Organic Corner" and "Clothing" sections to share high-quality material data.

| Field | Type | Description |

|---|---|---|

| id | UUID | Unique ID for the material. |

| material_name | String | e.g., "Handspun Mulberry Silk." |

| origin | String | e.g., "Rajshahi, Bangladesh." |

| quality_grade | String | "Premium," "Heritage," or "Organic." |

| texture_map | String | URL to the 3D texture for the AR viewer. |

| surcharge | Decimal | Extra cost for selecting this specific fabric. |

3. The Order & Progress Table (The Engine's Fuel)

This is the most critical table for your Python Progress Tracker.

| Field | Type | Description |

|---|---|---|

| order_id | UUID | Linked to the Customer's Gmail ID. |

| status | Enum | Pending, Artisan_Assigned, In_Progress, Shipped. |

| start_date | Timestamp | When the "Handmade" process actually began. |

| target_date | Timestamp | The estimated 14-day completion goal. |

| artisan_notes | Text | Hidden notes used by the AI to flavor the progress text. |

| is_paused | Boolean | Admin toggle to stop the AI progress bar if there's a delay. |

How it all connects

* The Admin posts a new design to Table 1.
* The Customer selects a fabric from Table 2 and orders.
* The Backend creates a row in Table 3.

* The Python Engine (which I gave you earlier) reads the start_date and target_date from Table 3 to calculate the non-linear percentage for the customer's dashboard.

* The Gemini Chatbot reads Tables 1 and 2 to answer questions like "Where does the silk for the Nilambari come from?"

To help you get Aurnik up and running, here is a sample JSON dataset. You can import these directly into your Supabase/PostgreSQL tables to test the AI Progress Engine and the Chatbot’s knowledge base.

1. Product Collection (Fixed Designs)

Table: products


```
[
  {
    "id": "d16b-4e12",
    "name": "The Midnight Jamdani",
    "base_price": 18500,
    "description": "A heritage-grade black Jamdani featuring hand-woven silver motifs. Each piece takes 120 hours of manual loom work.",
    "category": "Clothing",
    "ar_model_url": "https://res.cloudinary.com/aurnik/raw/upload/midnight_jamdani.glb",
    "is_limited": true,
    "total_allocation": 15
  },
  {
    "id": "e22c-9k33",
    "name": "Artisanal Forest Honey",
    "base_price": 2200,
    "description": "Wild-sourced, organic honey from the deep Sundarbans, filtered through traditional muslin.",
    "category": "Organic",
    "ar_model_url": null,
    "is_limited": false,
```

"total_allocation": 100
}
]

2. Material Library (Fabric & Quality)

Table: materials

[
{
"id": "f55-silk",
"material_name": "Rajshahi Mulberry Silk",
"origin": "Rajshahi, Bangladesh",
"quality_grade": "Heritage",
"texture_map": "https://cloudinary.com/textures/silk_normal.jpg",
"surcharge": 4500
},
{
"id": "g88-linen",
"material_name": "Organic Belgian Linen",
"origin": "Imported / Eco-Certified",
"quality_grade": "Premium",
"texture_map": "https://cloudinary.com/textures/linen_rough.jpg",
"surcharge": 2000
}
]

3. Order Mock Data (For Progress Testing)

Table: orders

Use this to test your Python script. Set the start_date to 5 days ago to see the non-linear progress.


```
[
  {
    "order_id": "ord-7721",
    "customer_gmail": "user@gmail.com",
    "status": "In_Progress",
    "start_date": "2026-03-12T10:00:00Z",
    "target_date": "2026-03-26T10:00:00Z",
    "artisan_notes": "Currently working on the silver border motifs. Fabric tension is perfect.",
    "is_paused": false
  }
]
```


Setting Up the BD Environment (Final Steps)

* Database: Log into Supabase, go to the "SQL Editor," and paste the schema I gave you previously to create the tables. Then use the "Import" tool to upload these JSON files.

* AI Studio: Paste the System Instructions and the Final Prompt into Google AI Studio. It will generate the Next.js frontend code.

* Local Testing: Use Vercel CLI to deploy your frontend and Render to host your Python script.

Integrating the Aurnik Python script with your Supabase database allows the progress bar to move from "simulated" to "data-driven" while still maintaining that non-linear, artisanal feel.

Here is how to bridge the two systems using the supabase-py library.

1. Install Dependencies

In your Python environment (on Render), you will need to install the client:

```
pip install supabase
```

2. The Database-Connected Engine

This script fetches the start_date and target_date directly from your Supabase table and calculates the progress.

```
import os

from supabase import create_client, Client

from datetime import datetime

import math


# Initialize Supabase Client

url: str = os.environ.get("SUPABASE_URL")

key: str = os.environ.get("SUPABASE_ANON_KEY")

supabase: Client = create_client(url, key)


def get_live_progress(order_id):

    # 1. Fetch order details from Supabase

    response = supabase.table("orders").select("*").eq("order_id", order_id).execute()


    if not response.data:

        return {"error": "Order not found"}


    order = response.data[0]


    # 2. Check for Manual Pause (Admin Control)

    if order['is_paused']:

        return {

            "percent": "Paused",

            "stage": "Quality Review in Progress",

            "note": "Our head artisan is performing a manual inspection."

        }
```

```
# 3. Calculate Time Logic
start_date = datetime.fromisoformat(order['start_date'].replace('Z', ''))
target_date = datetime.fromisoformat(order['target_date'].replace('Z', ''))
current_date = datetime.now()

total_duration = (target_date - start_date).total_seconds()
elapsed = (current_date - start_date).total_seconds()

# Normalized time (0 to 1)
x = max(0, min(1, elapsed / total_duration))

# 4. Apply Sigmoid Curve (The "Handmade" S-Curve)
# Stays longer in the 40%-70% range (the embroidery/hand-stitching phase)
progress_raw = 1 / (1 + math.exp(-6 * (x - 0.5)))

# 5. Mapping to Stages
percent = round(progress_raw * 100, 1)
stage = "Preparing Materials"
if percent > 20: stage = "Precision Cutting"
if percent > 40: stage = "Artisanal Hand-Stitching"
if percent > 85: stage = "Final Detailing & QC"
if percent >= 99: stage = "Luxury Packaging"

return {
    "percent": percent,
    "stage": stage,
    "artisan_note": order['artisan_notes']
```

```
    }

# Example Usage for your Dashboard API
# print(get_live_progress("ord-7721"))
```

3. Implementation Steps for the BD Environment

* Environment Variables: In your Render dashboard, add SUPABASE_URL and SUPABASE_ANON_KEY. This keeps your database credentials safe and out of the code.
* The API Route: Wrap the function above in a FastAPI @app.get("/progress/{order_id}") route.
* The Frontend Sync: In your Next.js frontend, use SWR or React Query to fetch this data every time the user opens their "Order Vault."

The "Aurnik" Launch Map

* Step 1: Use the Supabase Dashboard to manually add your first "Midnight Jamdani" order.
* Step 2: Deploy the Python Script to Render.
* Step 3: Use the AI Studio Prompt to generate your "Vault" page that calls this script.

Your Aurnik ecosystem is now technically complete. You have the luxury brand identity, the AR visualization logic, the AI concierge, and the artisanal progress engine.